Fase 4 — Cierre de la orden (Punto 4)

Rama: `fase-4-cierre-orden` · Implementación y pruebas: 31/10/2026 · Responsable: **Benjamín Alasino** · Estado: **completa y verificada en la rama** (pendientes en la sección 10).

Este archivo tiene dos partes:

- **Parte 1:** informe técnico de lo implementado y lo verificado.
- **Parte 2:** guía de estudio para la defensa oral (secciones A–J).

Convención de rutas: `SRC` = `src/main/java/ar/edu/iua/iw3`. El plan y la guía de inicio están en `docs/plan-de-trabajo.md` (sección 6.1) y `docs/fase-4-guia-de-inicio.md`.

Parte 1 — Informe técnico

1. Alcance y resultado

La fase implementa el **Punto 4 de la consigna**: "*Debe implementarse un endpoint que permita cerrar la orden, bloqueando la recepción de nuevos datos de detalle. Al ejecutarse este proceso, la orden pasa a estado 3: Cerrada para carga.*"

- **Endpoint nuevo:** `POST /api/v1/integration/loader/orders/{orderNumber}/close`, sin cuerpo, con JWT. Lo usa el sistema de carga cuando termina de cargar el camión.
- **Qué hace:** si la orden existe y está en estado 2, la pasa al estado 3, deja una fila en `order_status_logs` (2 → 3, usuario del token, fecha y hora, observación) y responde 200 con la orden.
- **Bloqueo de lecturas:** no hizo falta código nuevo. `OrderDetailBusiness.add` (Fase 3) ya respondía 409 a toda orden que no estuviera en estado 2; la Fase 4 crea la transición que activa ese bloqueo. Se verificó de punta a punta.

Resultado: los 9 criterios de aceptación están cumplidos y verificados con pruebas ejecutadas (sección 7). No se modificó código de las Fases 1 a 3. Se registran tres observaciones nuevas (H-23 a H-25, sección 9); ninguna bloquea la fase.

#	Criterio de aceptación	Resultado
CA-1	Cerrar una orden en estado 2: 200 y la orden en estado 3 (también en `GET /orders/{n}`)	Cumplido
CA-2	`order_status_logs` suma una fila 2→3 con el usuario del token, fecha y hora y la observación	Cumplido
CA-3	Una lectura tras el cierre: 409 y no se guarda (el histórico no cambia)	Cumplido
CA-4	Cerrar otra vez o en estado 1: 409; inexistente: 404; sin token: 401	Cumplido
CA-5	El cierre no cambia `loadingStartDateTime`, `loadingEndDateTime` ni los últimos valores	Cumplido
CA-6	Lecturas y cierre simultáneos sin 500 y cabecera = máximo guardado; dos cierres simultáneos: un 200 y un 409	Cumplido (dos corridas)
CA-7	`./mvnw -B test` y la colección Postman completa sin fallas	Cumplido (19 tests; 86 pedidos y 204 aserciones, 0 fallas)
CA-8	Swagger, informe y guía (≥ 10 preguntas), PDF, `CLAUDE.md`, plan y guía de `src` actualizados	Cumplido con este documento
CA-9	Cambios publicados en la rama `fase-4-cierre-orden`	Ver el resumen final de la fase (push)

2. Decisiones de diseño (29 a 32)

Las cuatro estaban propuestas en `docs/plan-de-trabajo.md` y se aplicaron sin cambios.

#	Decisión	Por qué
29	Solo se cierra desde el estado 2. Los estados 1, 3 y 4 responden **409**. Cerrar dos veces **no** es idempotente: la segunda da 409	Los cambios de estado son secuenciales y unidireccionales. A diferencia de la activación (decisión 19), repetir un cierre no es un reintento inocuo: indica un error del cliente y no debe sumar otra fila al historial
30	**A confirmar por el equipo.** El cierre no exige que la orden esté activada ni que tenga lecturas: alcanza con el estado 2	La consigna solo pide cerrar la orden; no menciona activación ni lecturas como condición. Permite cerrar una orden que se canceló antes de cargar
31	La orden se busca con `findByOrderNumberForUpdate` (`PESSIMISTIC_WRITE`)	`Order` no tiene `@Version` ni `@DynamicUpdate`: Hibernate escribe **todas** las columnas. Sin bloqueo, un cierre que leyó la orden antes de que confirme una lectura pisaría la cabecera con valores viejos. Con el bloqueo, cierre y lecturas de la misma orden se procesan de a uno
32	Una fila en `order_status_logs` (de 2 a 3, actor = usuario del token, observación fija `"Orden cerrada para carga"`) y respuesta 200 con la orden (la misma representación que `GET /orders/{n}`)	La consigna pide registrar cada cambio de estado con fecha, usuario y observaciones. La fecha y hora del cierre ya queda en el historial, por eso **no** se agrega un campo `closeDateTime` al modelo

Otras decisiones técnicas de la fase:

- **`saveAndFlush` en lugar de `save`:** obliga a Hibernate a ejecutar el `UPDATE` dentro del `try`. Si la base falla, sale como `BusinessException` con formato `StandartResponse` y no como un 500 de Spring al confirmar. Es la lección del hallazgo H-14 de la Fase 3.
- **Helper `loadForUpdate` duplicado:** `OrderBusiness` tiene su propio `loadForUpdate` privado, equivalente al de `OrderDetailBusiness`. Se duplicó a propósito para no refactorizar la Fase 3 (fuera de alcance). Queda como deuda menor (H-24).
- **Sin DTO de respuesta:** se devuelve la entidad `Order`, igual que `GET /orders/{n}`. `activationPassword` sigue oculta por `@JsonIgnore`.

3. Archivos de la fase

Archivo	Acción	Motivo	Commit
`SRC/business/IOrderBusiness.java`	Modificado	Firma de `close(Integer orderNumber, String actor)` con su Javadoc	`87388ba`
`SRC/business/OrderBusiness.java`	Modificado	Método `close` y helper privado `loadForUpdate`	`87388ba`
`SRC/integration/loader/controllers/OrderLoaderRestController.java`	Modificado	Endpoint `POST /orders/{orderNumber}/close` con anotaciones Swagger	`729d951`
`src/test/java/ar/edu/iua/iw3/business/OrderBusinessCloseTest.java`	Creado	7 tests unitarios del cierre con Mockito	`2722036`, `dbeb4c1`
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Modificado	Carpeta `04 - Fase 4: cierre (sistema de carga)`: 20 pedidos, 41 aserciones y 5 variables propias. Las carpetas 00 a 03 no cambian (el diff solo agrega líneas)	`e2a4d9f` … `0da2204` (11 commits)
`docs/fase-4-cierre-orden.md` y `docs/pdf/fase-4-cierre-orden.pdf`	Creados	Este informe y su guía	Commits `docs` de la fase
`CLAUDE.md`, `docs/plan-de-trabajo.md`, `docs/guia-estudio-src.md`, `docs/pdf/README.md`, `docs/pdf/guia-estudio-src.pdf`	Modificados	Estado del proyecto, decisiones 29 a 32 y guía del código	Commits `docs` de la fase

No se tocaron: `OrderDetailBusiness`, `OrderDetailValidator`, `Order`, `OrderRepository`, `Constants`, `SecurityConfiguration` ni el `pom.xml`. No hay tablas ni columnas nuevas.

4. Recorrido de `POST /api/v1/integration/loader/orders/{orderNumber}/close`

1. **Filtro JWT** (`JWTAuthorizationFilter`, de la cátedra). Sin token, o con un token inválido, responde **401** y la petición no llega al controller. La ruta no figura entre las públicas de `SecurityConfiguration`, así que la cubre `anyRequest().authenticated()`.
2. **Controller** — `OrderLoaderRestController.close(@PathVariable Integer orderNumber)`. Spring convierte `{orderNumber}` a `Integer`; si no es numérico, responde **400** antes de entrar al método (H-23). No hay `@RequestBody`: un cuerpo, si llega, se ignora. Toma el usuario con `getUserLogged()` y llama a `orderBusiness.close(orderNumber, username)`.
3. **Negocio** — `OrderBusiness.close`, dentro de una transacción (`@Transactional(rollbackFor = Exception.class)`):
    1. `loadForUpdate(orderNumber)` → `orderDAO.findByOrderNumberForUpdate` ejecuta `SELECT … FOR UPDATE` y **bloquea la fila** hasta el fin de la transacción. Una lectura o un cierre de la misma orden que llegue en ese momento espera. Si la consulta falla, `BusinessException` (500). Si no hay fila, `NotFoundException` (**404**), fuera del `try`.
    2. Si el estado no es `INITIAL_WEIGHING_REGISTERED` (2), `ConflictException` (**409**) con el estado actual en el mensaje, fuera del `try`.
    3. Dentro del `try`: guarda `from = 2`, pone el estado en `CLOSED_FOR_LOADING` (3), ejecuta `orderDAO.saveAndFlush(order)` (`UPDATE orders …`) y `logStatus(order, 2, 3, actor, "Orden cerrada para carga")` (`INSERT INTO order_status_logs …`).
    4. Si algo del paso 3 falla, `catch (Exception e)` → `BusinessException` y el *rollback* deshace las dos escrituras.
4. **Fin de la transacción:** `COMMIT`. Se liberan el bloqueo de la fila y quedan confirmados el estado 3 y la fila del historial.
5. **Respuesta:** el controller devuelve `200` con la orden. Las excepciones se traducen con `response.build(status, e, e.getMessage())` (formato `StandartResponse`): 404, 409 o 500.

Qué pasa después con una lectura. `OrderDetailBusiness.add` también toma el bloqueo de la fila. Si llega mientras el cierre está en curso, espera; cuando el cierre confirma, la lectura lee el estado 3 y responde **409**: "*La orden N no acepta lecturas: debe estar en estado 2 y activada (estado 3, activada=true)*". No escribe nada.

5. Funciones y métodos

`IOrderBusiness.close` — `SRC/business/IOrderBusiness.java`

`Order close(Integer orderNumber, String actor) throws NotFoundException, ConflictException, BusinessException`. Contrato del cierre, documentado con Javadoc: qué comprueba (existencia y estado 2), qué cambia (estado 3), qué escribe (historial) y qué **no** toca (fechas de carga, últimos valores).

`OrderBusiness.close` — `SRC/business/OrderBusiness.java`

Parámetros	`orderNumber`: número de la orden (viene del path). `actor`: usuario del token (puede ser `null`)
Retorno	La orden ya en estado 3 (la que devolvió `saveAndFlush`)
Validaciones	404 si no existe; 409 si el estado no es 2 (incluye cerrar dos veces). Las dos fuera del `try`
Dependencias	`orderDAO` (`OrderRepository`), `statusLogDAO` (a través de `logStatus`)

Errores	`NotFoundException` → 404 · `ConflictException` → 409 · `BusinessException` → 500 (falla técnica de la consulta, del `UPDATE` o del `INSERT`)
Transacción	`@Transactional(rollbackFor = Exception.class)`: las excepciones de negocio son *checked* y sin `rollbackFor` Spring no revertiría
No modifica	`loadingStartDateTime`, `loadingEndDateTime`, `lastAccumulatedMass`, `lastDensity`, `lastTemperature`, `lastFlowRate`, `activationPassword`, `activationDateTime`

`OrderBusiness.loadForUpdate` (privado, nuevo)

Busca con `findByOrderNumberForUpdate`. Envuelve la consulta en un `try` (un error técnico sale como `BusinessException`) y arma el 404 **fuera** del `try`, con el mismo texto que `load`: `"No se encuentra la Orden numero=N"`. Es equivalente al de `OrderDetailBusiness` (H-24).

`OrderBusiness.logStatus` (privado, existente, sin cambios)

Guarda una fila `OrderStatusLog(order, from, to, now, actor, observations)`. Si el actor es `null` o está en blanco, usa `"sistema"`.

`OrderLoaderRestController.close` — `SRC/integration/loader/controllers/OrderLoaderRestController.java`

`@PostMapping(value = "/orders/{orderNumber}/close", produces = APPLICATION_JSON_VALUE)`, sin `consumes` ni `@RequestBody`. Un `try/catch` por excepción, igual que el resto de los controllers. Swagger: `@Operation` y `@ApiResponse` para 200 (esquema `Order`), 401, 404, 409 y 500.

`OrderBusinessCloseTest` — `src/test/java/ar/edu/iua/iw3/business/OrderBusinessCloseTest.java`

JUnit 5 + Mockito (`@ExtendWith(MockitoExtension.class)`). `@Mock` de `OrderRepository` y `OrderStatusLogRepository`; `@InjectMocks OrderBusiness`. No usa la base. Detalle de los casos en la sección 7.1.

6. Relación con la consigna

Consigna	Dónde se cumple
"Un endpoint que permita cerrar la orden"	`POST /integration/loader/orders/{orderNumber}/close`
"bloqueando la recepción de nuevos datos de detalle"	La transición a 3 activa el 409 de `OrderDetailBusiness.add` (Fase 3); verificado en Postman y en la prueba de concurrencia
"la orden pasa a estado 3: Cerrada para carga"	`order.setStatus(OrderStatus.CLOSED_FOR_LOADING)`
Cambios de estado secuenciales y unidireccionales	Solo 2 → 3; desde 1, 3 o 4 responde 409 (decisión 29)
Cada cambio registrado: fecha y hora, usuario o proceso, observaciones	Fila en `order_status_logs` con `dateTime`, `actor` (usuario del token) y `"Orden cerrada para carga"`
Notas de estados: los datos de carga llegan en estado 2	Las lecturas solo se aceptan en estado 2; el diagrama del PDF rotula el 3 como "proceso de carga", pero el anexo y el requisito del final ("tiempo transcurrido, solo estado 2") confirman la interpretación vigente desde la Fase 3

7. Pruebas ejecutadas (31/10/2026)

Entorno: Windows 11, JDK 21, MySQL 8.0 en Docker (`iw3-gas-mysql`). Las pruebas de API se hicieron sobre una base **descartable** (`iw3_scratch`) con la app en el puerto 8089, para no ensuciar `iw3_db`; al terminar se borró.

7.1 Tests de Maven

Línea base **antes** de tocar código: `./mvnw -B test` → **12 tests, 0 fallas**. Con la fase: **19 tests, 0 fallas, 0 errores** (`BUILD SUCCESS`).

Test (`OrderBusinessCloseTest`)	Qué comprueba
`close_orderNotFound_throwsNotFound`	Orden inexistente → `NotFoundException` con "No se encuentra la Orden numero=..."; no guarda ni escribe historial
`close_status1_throwsConflict`	Estado 1 → `ConflictException` ("no se puede cerrar", "estado 1"); nada se escribe
`close_status3_alreadyClosed_throwsConflict`	Estado 3 (cerrar dos veces) → 409; nada se escribe
`close_status4_throwsConflict`	Estado 4 → 409; nada se escribe
`close_status2_movesTo3AndLogsOneRow`	Estado 2 → 3; `saveAndFlush` una vez; usa la consulta **con bloqueo** y nunca `findByOrderNumber`; **una** fila de log con from 2, to 3, actor `admin`, observación y fecha
`close_nullOrBlankActor_logsSistema`	Actor `null` o `" "` → el log queda con `"sistema"`
`close_doesNotChangeLoadingDatesNorLastValues`	Las fechas de carga, los `last*` y la contraseña quedan iguales

7.2 Postman con newman

`npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json --env-var baseUrl=http://localhost:8089` → **86 pedidos, 204 aserciones, 0 fallas**. Las carpetas 00 a 03 (66 pedidos, 163 aserciones) siguen igual; la carpeta 04 suma 20 pedidos y 41 aserciones. Las aserciones de mensajes usan fragmentos (`include`), nunca el texto completo.

Nota de honestidad: la línea base de newman (66 y 163) **no** se corrió antes de cambiar el código, porque la máquina no tenía Node.js. Se instaló durante la fase y la corrida completa se hizo con el código nuevo; las carpetas 00 a 03 no se modificaron y dieron los mismos números esperados.

Pedidos de la carpeta 04	Esperado y obtenido
Crear orden, tara y activación de la orden de cierre	201, 200, 200
Dos lecturas válidas (500 y 1200 kg)	200, `saved: true`
Cabecera antes del cierre	Estado 2 y `lastAccumulatedMass` 1200; se guardan fechas y últimos valores en `closeHeaderBefore`
Cerrar la orden	200, estado 3, mismo número, sin `activationPassword`
Consultar la orden cerrada	Estado 3; fechas de carga y `last*` **iguales** a los guardados
Historial de estados	3 filas; la última de 2 a 3, actor `admin`, observación con "cerrada" y fecha
Lectura después del cierre	409 con "no acepta lecturas"
Histórico de lecturas	`totalElements` 2 (la rechazada no se guardó)
Activar la orden cerrada · cerrar otra vez	409 · 409 con "no se puede cerrar"
Crear orden en estado 1 y cerrarla	201 · 409 con "estado 1"

Pedidos de la carpeta 04	Esperado y obtenido
Orden inexistente (`1999999999`) · sin token	404 con "No se encuentra" · 401
Decisión 30: crear, tara y cerrar sin activar ni lecturas	201, 200, 200 con estado 3 y `activationDateTime`, `loadingStartDateTime` y `lastAccumulatedMass` en `null`

7.3 Concurrencia (CA-6)

Script descartable de bash y `curl`, fuera del repo. Una orden activada con 3 lecturas previas; luego **50 lecturas** de masa creciente disparadas en paralelo y un **cierre** lanzado en medio de ellas. Se corrió **dos veces**:

Medición	Corrida 1	Corrida 2
Respuestas de las lecturas	24 × 200, 26 × 409	25 × 200, 25 × 409
Respuesta del cierre	200	200
Respuestas 500	0	0
Estado final	3	3
`last_accumulated_mass` = `MAX(accumulated_mass)` de `order_details`	1240 = 1240	1250 = 1250
Filas 2→3 en el historial	1	1
Lecturas con `reading_timestamp` posterior a la fecha del cierre	0	0
`loading_end_date_time` ≤ fecha del cierre	Sí	Sí
Dos cierres simultáneos de otra orden en estado 2	Un 200 y un 409; 1 fila 2→3	Un 200 y un 409; 1 fila 2→3

El log de la app no registró ningún `ERROR` ni ningún *deadlock*. En la corrida 1, `order_details` quedó con 26 filas (3 previas + 23 nuevas): una de las 24 lecturas con 200 fue un descarte (`saved: false`), porque al serializarse en otro orden llegó con una masa menor que la anterior; es el comportamiento de la Fase 3. En la corrida 2 quedaron 28 filas (3 + 25): ningún descarte.

7.4 Casos límite (a mano, con `curl`)

Caso	Resultado
`POST …/orders/abc/close` (número no numérico)	**400** con el formato por defecto de Spring (`status`, `error`, `message: "Method parameter 'orderNumber': Failed to convert…"`), no `StandartResponse` (H-23)
`Content-Type: application/json` sin cuerpo	200; el cuerpo no se lee
`Content-Type: application/json` con `{}` sobre una orden cerrada	409; el cuerpo se ignora
`GET …/close`	405 (solo existe `POST`)
Cierre de una orden con lecturas / sin lecturas	200 en los dos casos (Postman y prueba manual)

7.5 Swagger

`GET /v3/api-docs` publica `POST /api/v1/integration/loader/orders/{orderNumber}/close` en el grupo "Integración sistema de carga": sin `requestBody`, con seguridad `bearerAuth`, el 200 con el esquema `Order` y las respuestas 401, 404, 409 y 500. `/swagger-ui.html` responde 200. La interfaz gráfica no se recorrió a mano: se comprobó el documento OpenAPI, que es lo que la interfaz muestra.

8. Cómo probarlo

Con Postman: importar `docs/postman/IW3-Gas-Terminal.postman_collection.json`, ejecutar `00 - Autenticación` y después la carpeta `04 - Fase 4: cierre (sistema de carga)`. Cada carpeta crea sus propias órdenes, así que se puede correr sola. Por consola: `npx newman@6 run docs/postman/IW3-Gas-Terminal.postman_collection.json`.

Con `curl` (app en `localhost:8080`, usuario de desarrollo `admin/admin123`):

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/login -d "username=admin&password=admin123")

# Orden 90001 ya creada (SAP), con tara (TMS) y, opcionalmente, activada y con lecturas
curl -s -X POST http://localhost:8080/api/v1/integration/loader/orders/90001/close \
     -H "Authorization: Bearer $TOKEN"
# 200 → la orden con "status" : 3

curl -s -X POST http://localhost:8080/api/v1/integration/loader/orders/90001/details \
     -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
     -d '{"accumulatedMass": 2000, "density": 0.52, "temperature": 18.4, "flowRate": 3600}'
# 409 → "La orden 90001 no acepta lecturas: debe estar en estado 2 y activada (estado 3, activada=true)"

curl -s -X POST http://localhost:8080/api/v1/integration/loader/orders/90001/close \
     -H "Authorization: Bearer $TOKEN"
# 409 → "La orden 90001 no se puede cerrar: debe estar en estado 2 (estado 3)"

curl -s http://localhost:8080/api/v1/orders/90001/status-log -H "Authorization: Bearer $TOKEN"
# [..., { "fromStatus" : 2, "toStatus" : 3, "actor" : "admin", "observations" : "Orden cerrada para carga", ...
}]
```

Comprobación en la base: `SELECT from_status, to_status, actor, observations, date_time FROM order_status_logs WHERE id_order = (SELECT id FROM orders WHERE order_number = 90001);`

9. Hallazgos y limitaciones

ID	Tipo	Descripción	Evidencia	Estado
H-23	Observación	Un `orderNumber` no numérico en la URL responde 400 con el formato por defecto de Spring, no con `StandartResponse`. Pasa igual en todos los endpoints con `{orderNumber}` desde la Fase 2	Sección 7.4	No se cambia (comportamiento por defecto de Spring; el TP no tiene manejador global de errores)
H-24	Deuda menor	`loadForUpdate` está duplicado en `OrderBusiness` y `OrderDetailBusiness`	Código	Se duplicó a propósito para no refactorizar la Fase 3. Se puede unificar en un cambio aparte, con acuerdo del equipo
H-25	Limitación (análisis de código, **no reproducida**)	`OrderBusiness.activate` (Fase 3) busca la orden **sin** bloqueo y guarda con `saveAndFlush`, que escribe todas las columnas. Si la **primera** activación de una orden coincide con su cierre, la activación podría escribir el estado 2 que leyó antes y deshacer el estado 3, mientras el historial dice 2 → 3. Solo es posible con la decisión 30 (cerrar una orden no activada) y una activación en el mismo instante	Análisis de código	Aceptado para el parcial, como H-16. Arreglo propuesto para el equipo: que `activate` también use `findByOrderNumberForUpdate`. Es un cambio en la Fase 3, fuera del alcance de esta fase

Siguen vigentes las limitaciones anteriores que afectan al cierre: el token vence en 1 h (H-18) y no hay roles (cualquier usuario autenticado puede cerrar una orden; los roles son del examen final).

10. Pendientes

- **Decisión 30, a confirmar por el equipo:** si se decide que el cierre exija la activación, el cambio es una condición más en `OrderBusiness.close` (409), un test y un pedido de Postman.
- **H-25:** decidir si se agrega el bloqueo a `activate` (cambio de la Fase 3).
- **Pull Request a `main`:** lo abre Benjamín (o lo pide). Por la decisión 14 necesita la aprobación de 3 de los 4 integrantes.
- **Fase 5:** pesaje final (estado 3 → 4) y conciliación. No se empieza sin autorización.

Parte 2 — Guía de estudio de la Fase 4

A. Objetivo de la fase

Que el sistema de carga pueda **dar por terminada la carga** de una orden. Al cerrarla, la orden pasa del estado 2 (*con pesaje inicial registrado*, donde se reciben las lecturas) al estado 3 (*cerrada para carga*). Desde ese momento el backend **no acepta más lecturas** del caudalímetro, y la orden queda lista para el pesaje final (Fase 5). El cambio queda auditado en el historial de estados.

B. Conceptos teóricos

1. Máquina de estados (dominio de la consigna)

La orden recorre 1 → 2 → 3 → 4, **de a un paso y sin volver atrás**. Cada operación solo es válida en un estado: la tara en el 1, la activación y las lecturas en el 2, el cierre en el 2 y el pesaje final en el 3. Implementarlo es simple: antes de cambiar nada, se compara el estado actual con el esperado y, si no coincide, se rechaza. El enum `OrderStatus` guarda el número (1 a 4) en la base y en el JSON.

2. Transacciones y excepciones *checked* (clases 3 y 7 de la cátedra)

`@Transactional` hace que todo el método sea una sola unidad: el `UPDATE` de la orden y el `INSERT` del historial se confirman juntos o no se confirma ninguno. Por defecto, Spring solo revierte ante excepciones *unchecked* (`RuntimeException`). Las de negocio de la cátedra son *checked*: por eso el TP usa `rollbackFor = Exception.class`.

3. Bloqueo pesimista (complementario: la cátedra no lo usa)

`@Lock(LockModeType.PESSIMISTIC_WRITE)` hace que la consulta sea `SELECT … FOR UPDATE`: MySQL **bloquea la fila** hasta que termina la transacción. Otra transacción que quiera la misma fila con `FOR UPDATE` espera su turno. Así un cierre y una lectura de la misma orden no se mezclan: o la lectura entra antes (y el cierre ve la cabecera ya actualizada) o entra después (y ve el estado 3 → 409). El bloqueo **optimista** (`@Version`) detecta el choque al guardar y obliga a reintentar; acá se eligió el pesimista porque ya lo usaban las lecturas (hallazgo H-15 de la Fase 3).

4. `save` frente a `saveAndFlush` (complementario)

`save` deja el `UPDATE` pendiente hasta el `COMMIT`, que ocurre **después** de que el método terminó, fuera del `try/catch`. Si la base falla en ese momento, Spring responde un 500 con su propio formato. `saveAndFlush` ejecuta el `UPDATE` en el acto, dentro del `try`, y el error sale como `BusinessException` con el formato de la cátedra (lección de H-14).

5. Códigos HTTP: 409 frente a 400 (complementario: la cátedra usa 404, 302 y 500)

- **400 Bad Request:** la petición está mal formada (un número que no es número, un campo obligatorio ausente). Repetirla igual nunca va a funcionar.
- **409 Conflict:** la petición es correcta, pero choca con el **estado actual** del recurso (la orden ya está cerrada, o todavía no tiene tara). Si el estado cambiara, la misma petición podría funcionar.
- **404:** la orden no existe. **401:** falta el token o no es válido (lo responde el filtro de seguridad).

6. Idempotencia (complementario)

Una operación es idempotente si repetirla deja el sistema igual que hacerla una vez. La activación lo es (decisión 19). El cierre **no** (decisión 29): la segunda llamada responde 409 y no agrega otra fila al historial.

C. Arquitectura

```
Sistema de carga ── POST /api/v1/integration/loader/orders/{n}/close + JWT
        │
        ▼
JWTAuthorizationFilter ── sin token → 401
        │
        ▼
OrderLoaderRestController.close  (getUserLogged → actor; try/catch → 404 / 409 / 500)
        │
        ▼
IOrderBusiness.close → OrderBusiness.close   @Transactional(rollbackFor = Exception.class)
        │   loadForUpdate ──▶ OrderRepository.findByOrderNumberForUpdate  (SELECT … FOR UPDATE)
        │   estado ≠ 2 → ConflictException
        │   setStatus(3) ──▶ orderDAO.saveAndFlush  (UPDATE orders)
        │   logStatus ─────▶ statusLogDAO.save      (INSERT order_status_logs)
        ▼
MySQL: orders.status = 3 · nueva fila en order_status_logs · COMMIT libera la fila
```

Las capas son las de la cátedra: el controller solo traduce HTTP, el negocio tiene las reglas y el repositorio habla con la base.

D. Código paso a paso

1. El negocio (`OrderBusiness.close`)

```
@Override
@Transactional(rollbackFor = Exception.class)
public Order close(Integer orderNumber, String actor) throws NotFoundException, ConflictException,
BusinessException {
    Order order = loadForUpdate(orderNumber);                          // (1) 404 fuera del try, con bloqueo
    if (order.getStatus() != OrderStatus.INITIAL_WEIGHING_REGISTERED) { // (2) 409 fuera del try
        throw ConflictException.builder().message("La orden " + orderNumber
                + " no se puede cerrar: debe estar en estado 2 (estado " + order.getStatus().getCode() +
")").build();
    }
    try {
        OrderStatus from = order.getStatus();                          // (3) se recuerda el 2 para el historial
        order.setStatus(OrderStatus.CLOSED_FOR_LOADING);               // (4) 2 → 3
        order = orderDAO.saveAndFlush(order);                          // (5) UPDATE inmediato (H-14)
        logStatus(order, from, order.getStatus(), actor, "Orden cerrada para carga"); // (6) auditoría
        return order;
    } catch (Exception e) {                                            // (7) falla técnica → 500 y rollback
        log.error(e.getMessage(), e);
        throw BusinessException.builder().ex(e).build();
    }
}
```

- **(1) y (2) fuera del `try`:** si estuvieran adentro, `catch (Exception e)` atraparía el 404 y el 409 y los convertiría en 500.
- **(3)** se guarda el estado anterior **antes** de cambiarlo; si no, el historial diría "de 3 a 3".
- **(6)** `logStatus` pone `"sistema"` si el actor llega vacío.
- **No** se tocan `loadingEndDateTime` ni los `last*`: el fin de carga es la estampa del último registro válido (decisión 25), no la del cierre.

2. El controller (`OrderLoaderRestController.close`)

```
@PostMapping(value = "/orders/{orderNumber}/close", produces = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<?> close(@PathVariable Integer orderNumber) {
    try {
        User user = getUserLogged();                                    // usuario del JWT
        Order order = orderBusiness.close(orderNumber, user == null ? null : user.getUsername());
        return new ResponseEntity<>(order, HttpStatus.OK);
    } catch (NotFoundException e) { … 404 … }
      catch (ConflictException e) { … 409 … }
      catch (BusinessException e) { … 500 … }
}
```

Sin `@RequestBody` ni `consumes`: el cierre no necesita datos, solo el número de orden de la URL. `getUserLogged()` viene de `BaseRestController` (decisión 12).

3. El bloqueo de lecturas (código de la Fase 3, sin cambios)

`OrderDetailBusiness.add` ya tenía esta condición: `if (order.getStatus() != INITIAL_WEIGHING_REGISTERED || order.getActivationDateTime() == null) → ConflictException`. Al pasar la orden a 3, todas las lecturas siguientes caen ahí. La Fase 3 lo había anticipado forzando `status = 3` con SQL (sección 7.6 de su informe); la Fase 4 lo comprobó con el endpoint real.

E. Trazabilidad de los datos

Dato	Origen	Dónde queda	Quién lo cambia en el cierre
`orderNumber`	Path de la URL	Solo para buscar la orden	Nadie
Usuario	JWT (`getUserLogged()`)	`order_status_logs.actor`	`logStatus`
Estado	`OrderStatus.CLOSED_FOR_LOADING`	`orders.status` = 3	`OrderBusiness.close`
Fecha y hora del cierre	`LocalDateTime.now()` en el servidor	`order_status_logs.date_time`	`logStatus`
Observación	Texto fijo `"Orden cerrada para carga"`	`order_status_logs.observations`	`logStatus`
Estado anterior	`order.getStatus()` antes del cambio (2)	`order_status_logs.from_status`	`OrderBusiness.close`
Fechas de carga y últimos valores	Lecturas de la Fase 3	`orders.loading_*`, `orders.last_*`	**Nadie**: el cierre no los toca

F. Ejemplo práctico de punta a punta

1. SAP crea la orden 90001 → estado 1 (historial: `null → 1`).
2. La balanza registra la tara → estado 2 y contraseña 48213 (historial: `1 → 2`).
3. El sistema de carga se activa con 48213 → recibe el preset 15000 kg (estado 2).

4. Llegan lecturas: 500 kg, 1200 kg… La cabecera queda con `lastAccumulatedMass` 1200 y `loadingEndDateTime` = la hora de la última lectura.
5. **Cierre:** `POST /api/v1/integration/loader/orders/90001/close` → 200 con `"status" : 3`. El historial suma `2 → 3`, actor `admin`, "Orden cerrada para carga".
6. Llega una lectura tarde (1500 kg) → 409 "no acepta lecturas"; `order_details` sigue con 2 filas y la cabecera sigue en 1200.
7. Se intenta cerrar otra vez → 409 "no se puede cerrar … (estado 3)".
8. Próximo paso (Fase 5): la balanza envía el pesaje final y la orden pasa a 4.

G. Preguntas del profesor

1. ¿Qué pide exactamente el Punto 4 de la consigna y cómo lo resolvieron?
2. ¿Por qué el bloqueo de lecturas no necesitó código nuevo?
3. ¿Por qué la comprobación del estado está fuera del `try`?
4. ¿Por qué usan `@Transactional(rollbackFor = Exception.class)` y qué se revierte si algo falla?
5. ¿Qué pasa si una lectura y el cierre llegan al mismo tiempo? ¿Por qué usan `findByOrderNumberForUpdate`?
6. ¿Por qué `saveAndFlush` y no `save`?
7. ¿Por qué responden 409 y no 400 cuando la orden ya está cerrada?
8. ¿Es idempotente el cierre? ¿Por qué la activación sí y el cierre no?
9. ¿Dónde queda la fecha y hora del cierre? ¿Por qué no agregaron un campo `closeDateTime`?
10. ¿Por qué el cierre no modifica `loadingEndDateTime`?
11. ¿Se puede cerrar una orden que nunca se activó? (decisión 30)
12. ¿Cómo se sabe quién cerró la orden?
13. ¿Cómo probaron el cierre sin base de datos? ¿Qué es Mockito?
14. ¿Cómo comprobaron la concurrencia y qué resultados obtuvieron?
15. ¿Qué limitaciones tiene la fase?

H. Respuestas modelo

1. **Punto 4.** Un endpoint que cierre la orden, bloquee nuevos datos de detalle y la pase a estado 3. Lo resolvimos con `POST /api/v1/integration/loader/orders/{n}/close`: comprueba que la orden exista (404) y esté en estado 2 (409), la pasa a 3 y registra el cambio en `order_status_logs`.
2. **Bloqueo sin código nuevo.** Desde la Fase 3, `OrderDetailBusiness.add` rechaza con 409 cualquier lectura de una orden que no esté en estado 2. Faltaba solo la transición 2 → 3, que es lo que agrega el cierre. Lo verificamos en Postman y con lecturas en paralelo.
3. **Fuera del `try`.** El `catch (Exception e)` convierte todo en `BusinessException` (500). Si el `throw` del 409 estuviera adentro, el cliente recibiría 500 en lugar de 409.
4. **Transacción.** El `UPDATE` de la orden y el `INSERT` del historial tienen que quedar juntos. Como nuestras excepciones son *checked*, Spring no revertiría sin `rollbackFor = Exception.class`. Si falla el `INSERT`, la orden vuelve al estado 2.
5. **Concurrencia.** `findByOrderNumberForUpdate` ejecuta `SELECT … FOR UPDATE` y bloquea la fila. Las lecturas usan el mismo bloqueo, así que se procesan de a una. Si la lectura llega antes, el cierre espera y ve la cabecera actualizada; si llega después, ve el estado 3 y da 409. Sin el bloqueo, como Hibernate escribe todas las columnas, el cierre podría pisar la cabecera con valores viejos.
6. **`saveAndFlush`.** Ejecuta el `UPDATE` dentro del `try`. Con `save`, un error de base aparecería al confirmar, fuera de nuestro código, y saldría como 500 de Spring sin el formato `StandartResponse` (hallazgo H-14 de la Fase 3).

7. **409 y no 400.** La petición está bien formada; el problema es el estado de la orden. 400 es para datos inválidos (por ejemplo, un número de orden que no es número).
8. **Idempotencia.** La activación devuelve siempre el mismo preset y no cambia nada la segunda vez: reintentarla es inofensivo. Cerrar una orden ya cerrada no es un estado válido de partida (los cambios son unidireccionales) y no debe sumar otra fila al historial; por eso responde 409 (decisión 29).
9. **Fecha del cierre.** En `order_status_logs.date_time`, en la fila 2 → 3. Agregar un campo duplicaría el dato; la consigna pide registrar cada cambio de estado, y eso ya lo hace el historial (decisión 32).
10. **`loadingEndDateTime`.** Es la estampa de la última lectura válida (decisión 25), que es lo que pide la consigna como "fin de carga". El cierre puede ocurrir minutos después.
11. **Decisión 30.** Sí: alcanza con el estado 2, porque la consigna no pide otra cosa. Sirve, por ejemplo, para cerrar una orden cancelada antes de cargar. Queda a confirmar por el equipo.
12. **Quién cerró.** El controller toma el usuario del JWT con `getUserLogged()` y lo pasa como `actor`; queda en `order_status_logs.actor`. Si llega vacío, se guarda `"sistema"`.
13. **Mockito.** Crea dobles de los repositorios (`@Mock`) y los inyecta en `OrderBusiness` (`@InjectMocks`). Programamos qué devuelven (`when(...).thenReturn(...)`) y verificamos qué se llamó (`verify`, `ArgumentCaptor`). Así probamos las reglas sin base: 7 tests.
14. **Concurrencia probada.** Disparamos 50 lecturas en paralelo y un cierre en medio, dos veces. Resultado: ningún 500, una sola fila 2 → 3, ninguna lectura guardada después del cierre y la masa de la cabecera igual al máximo guardado. Dos cierres simultáneos dieron un 200 y un 409.
15. **Limitaciones.** H-23 (número no numérico: 400 con formato de Spring), H-24 (helper duplicado), H-25 (una primera activación simultánea con el cierre podría deshacerlo; no reproducida, aceptada como H-16), y sin roles (cualquier usuario autenticado puede cerrar).

I. Errores frecuentes

1. **Poner el `throw` del 409 dentro del `try`:** el cliente recibe 500 en lugar de 409.
2. **Buscar la orden con `load` (sin bloqueo):** compila y pasa las pruebas secuenciales, pero con lecturas simultáneas el cierre puede pisar la cabecera con valores viejos.
3. **Olvidar `rollbackFor = Exception.class`:** si falla el historial, la orden queda en estado 3 sin registro del cambio.
4. **Guardar el estado anterior después de cambiarlo:** el historial queda "de 3 a 3".
5. **Actualizar `loadingEndDateTime` al cerrar:** mezcla el fin de carga (última lectura válida) con el momento del cierre.
6. **Hacer el cierre idempotente (200 la segunda vez):** suma filas repetidas al historial y oculta un error del cliente.
7. **Pedir `@RequestBody` o `consumes = JSON`:** el sistema de carga tendría que mandar un cuerpo que no se usa. Con `consumes`, un pedido sin `Content-Type: application/json` daría 415; con un `@RequestBody` obligatorio, uno sin cuerpo daría 400.
8. **Tocar `OrderDetailBusiness` para "bloquear" las lecturas:** el bloqueo ya existe; basta con la transición.
9. **Comparar mensajes completos en Postman:** un cambio de redacción rompe la prueba; se usan fragmentos con `include`.
10. **Reutilizar las variables de la carpeta 03 en la 04:** las carpetas dejan de ser independientes. La 04 usa las suyas (`orderNumberClose`, `activationPasswordClose`, `closeHeaderBefore`, `orderNumberClosePending`, `orderNumberCloseBare`).

J. Lista de comprobación para rendir

- [ ] Sé citar el Punto 4 y decir qué endpoint lo cumple, con su método, su ruta y sus respuestas (200, 401, 404, 409, 500).
- [ ] Puedo recorrer la petición: filtro JWT → controller → `OrderBusiness.close` → repositorio → MySQL.

- [ ] Explico el orden de comprobaciones (404 → 409) y por qué van fuera del `try`.
- [ ] Explico `@Transactional(rollbackFor = Exception.class)` y qué se revierte.
- [ ] Explico el bloqueo pesimista y qué pasa con una lectura que llega durante el cierre.
- [ ] Explico `saveAndFlush` frente a `save` (H-14).
- [ ] Distingo 409 de 400 y de 404.
- [ ] Defiendo las decisiones 29 a 32 y sé que la 30 está a confirmar por el equipo.
- [ ] Sé dónde queda la fecha del cierre y quién lo hizo (`order_status_logs`).
- [ ] Sé por qué el cierre no toca las fechas de carga ni los últimos valores.
- [ ] Puedo correr `./mvnw -B test` (19 tests) y la carpeta 04 de Postman, y explicar qué prueba cada caso.
- [ ] Puedo contar la prueba de concurrencia y sus resultados.
- [ ] Conozco las limitaciones H-23 a H-25.